PUNJABI UNIVERSITY, PATIALA

Department of Computer Science & Engineering

FindIt

Smart campus lost & found — report, match, and reunite belongings

MINOR PROJECT | WEB DEV / FULL STACK

Presented by Taran Singh (12401067) · Prince (12401062)

B.Tech CSE

Under the guidance of Dr. Harpreet Kaur

Problem Statement

1

The Real-World Challenge

Lost items are announced in scattered chats and notice boards.

Posts are hard to search and easy to miss.

Private identifying details often end up in public threads.

2

The Need for a Simple Tool

One searchable place for lost and found reports.

Explainable match suggestions between opposite-type reports.

Claims and moderation that keep ownership proof private.

Project Objectives

✓

Prepare Trustworthy Records

Collect and moderate lost/found reports suitable for campus search and recovery.

✓

Learn Useful Similarities

Score opposite-type reports with category, text, location, and date signals — and explain why.

✓

Make It Accessible

Ship a Next.js app anyone can use: browse, report, claim, and administer without calling the API by hand.

Data Model & Entities

Five MongoDB collections power the recovery workflow

Entity	Type	Example	What it represents
User	Document	asha.menon@campus.example	Student or administrator account
Session	Document	hashed cookie id	Signed-in browser session
Item	Document	Lost earbuds · approved	Lost or found report
Claim	Document	pending → approved	Ownership request on a found item
ActivityLog	Document	approve_item	Moderation and claim history

How Each Signal Relates to Recovery

Why these four match inputs were chosen

Signal	Represents	Expected link	Why it matters
Category	Item class	Same class pairs are likelier	Strong, simple gate
Text	Title + description words	Objects are named similarly	Catches shared keywords
Location	Place words	Loss/find share a building	Campus geography hint
Date	Event proximity	Same-week events fit better	Decays outside 14 days

Data & Request Preparation

A campus tool is only as good as its validated records

a Validate Inputs

Zod schemas check enums, emails, dates, and upload types before persistence.

b Choose Status Path

Reports start pending; only approved items appear on public browse.

c Seed vs Live

npm run seed upserts fictional demo data; live demos use the same rules.

d Bound Queries

Match candidates are limited by type, status, category/date window, and caps.

Approaches Explained

Three ways to build campus recovery tools, in plain language

1

Plain Public CRUD

Anyone posts listings immediately. Easy to build, weak on spam and proof privacy.

2

Moderation Only

Admin approves posts, but no structured matching or claim workflow.

3

FindIt Path

Moderated listings + explainable matches + private claims. Used in this project.

Technology Stack

Next.js / React

App Router interface, TypeScript, Tailwind

Express

REST API with controllers, services, repositories

MongoDB / Mongoose

Document persistence and indexes

Zod / RHF

Request and form validation

Vitest / Supertest

Automated API tests in-memory

HTTP-only cookie

Session auth without localStorage tokens

System Architecture

Three clearly separated layers keep the app simple to build and explain

1

Presentation

Next.js pages and forms call /backend with credentials.

2

API Layer

Express validates, authorises, and applies claim/match rules.

3

Data Layer

Repositories talk to MongoDB; browser never sees the URI.

Design principle: keeping UI, rules, and persistence separate makes each part easy to test and keeps demos reliable.

Live App Demo: Browse & Report

Home shows approved lost / found / resolved counts.

Browse filters by type, category, location, and date.

Report form creates a pending item (optional image).

Pending reports stay off public browse until approved.

What's Happening Here

The student lands on a searchable campus workspace instead of a chat thread.

Creating a report is a form — not a public message dump.

Live App Demo: Match Explanation

Open a lost earbuds report from seed data.

The panel lists ranked found candidates.

Each row shows a score and plain-language reasons:

- Same category
- Shared words
- Similar location words
- Days apart

Disclaimer: heuristic estimate, not a probability.

What's Happening Here

Reviewers can see why two reports were linked before trusting a claim.

Weak matches can still be rejected after reading the proof privately.

Validation & Demo Success

Checked with automated API tests and a seeded walkthrough — not ML hold-out metrics

Vitest	API behaviours covered with Supertest + in-memory MongoDB
Privacy	Public JSON omits identifying details and claim proof
Demo path	Seed → match → claim → resolve works end-to-end

Which Signal Matters Most?

Default FindIt match weights

≈70%

of the default weight sits on category + text overlap

Category ≈ 35%

Text ≈ 35%

Location ≈ 15%

Date ≈ 15%

Minimum score to show: 0.35

Sample Walkthrough

The exact seeded scenario used in the demo, and what the system does

Seed

Demo students + earbuds reports

Match

Lost ↔ found with reasons

Claim

Proof visible to reviewers only

Resolve

Item marked resolved

Predicted outcome

A recoverable campus loop: searchable listings, explainable matches, moderated claims — not financial advice, not a chat dump.

Conclusion & Future Work

What We Achieved

- Full-stack FindIt with Next.js + Express + MongoDB
- Explainable match weights (35/35/15/15)
- Private claim proof + admin moderation
- Seeded demo path and automated tests

Future Work

- Email / in-app notifications on claim review
- Map or building list for locations
- Object storage for images
- Richer administrator audit views

Thank You

Questions are welcome

Taran Singh | 12401067 · Prince | 12401062

B.Tech CSE | Punjabi University, Patiala

Under the guidance of Dr. Harpreet Kaur